**VELLORE INSTITUTE OF TECHNOLOGY (VIT), VELLORE**

School of Advanced Sciences • Department of Mathematics • Winter Semester 2026

## BAMAT207 — PROBABILITY AND STATISTICS

# HamSpam: Probabilistic SMS Spam Detection

Discrete Probability Modeling, Bayes' Theorem Verification, and Multinomial Naive Bayes Classification

**Executive Project Abstract:** This academic report presents an end-to-end mathematical and empirical investigation of probabilistic text classification applied to the official UCI SMS Spam Collection benchmark (Dataset ID #228). We formalize SMS classification using discrete random variable theory, Bernoulli distributions, and Bayes' Theorem. Following strict preprocessing and deduplication (reducing 5,574 raw records to 5,171 distinct messages), we empirically verify Bayes' Law across targeted spam keywords (*free*, *prize*, *win*, *urgent*, *offer*), demonstrating identity between direct frequency ratios and inverted Bayesian posterior calculations to within machine epsilon ($\leq 2.22 \times 10^{-16}$). A Multinomial Naive Bayes classifier with Laplace smoothing ($\alpha = 1.0$) was trained on an 80:20 stratified split, achieving **98.07% Accuracy**, **95.12% Precision**, **89.31% Recall**, and an **F1-Score of 92.13%** on 1,035 held-out test messages. The complete system is deployed as a production-grade full-stack web application with FastAPI, React 18, and interactive telemetry.

| Overall Accuracy | Spam Precision | Spam Recall | Spam F1-Score |
|---|---|---|---|
| **98.07%** | **95.12%** | **89.31%** | **92.13%** |
| 1,015 / 1,035 test correct | Ultra-low FP (6 messages) | 117 / 131 spam detected | Harmonic mean purity |

| | | | |
|---|---|---|---|
| **Course Code & Title:** | BAMAT207 — Probability and Statistics | **Official Benchmark:** | UCI SMS Spam Collection (ID: 228) |
| **Candidate Name:** | Sarvesh S [Student Candidate] | **Live Cloud URL:** | https://hamspam.vercel.app |
| **Registration Number:** | 23BCE0000 [Editable Student ID] | **GitHub Repository:** | github.com/SarveshS1407/HamSpam |
| **Program & Department:** | B.Tech Computer Science / SAS & SCOPE | **Automated Test Suite:** | **17 / 17 Tests Passing (100%)** |
| **Faculty Supervisor:** | [Faculty Guide / Course Instructor] | **Submission Date:** | 27 October 2026 |

**Reproducibility & Verification Guarantee:** All statistical values, empirical proportions, confusion matrices, and evaluation scores presented in this report were generated directly by executing the automated project pipeline (src/probability_analysis.py, src/train_model.py) on the verified UCI SMS dataset. Zero synthetic data or speculative figures have been used.

# 1. Executive Summary & Problem Formulation

## 1.1 Executive Overview & Problem Statement

Short Message Service (SMS) remains one of the world's most ubiquitous and high-engagement direct communication channels, transmitting critical notifications including multi-factor authentication codes, financial transaction alerts, and personal correspondence. However, its open protocol and near-universal open rates have made it a lucrative target for unsolicited commercial messaging, deceptive phishing campaigns ('smishing'), and financial fraud. Unlike email systems that benefit from rich headers, DNS routing signatures, and extensive message length, SMS messages are restricted to 160 characters, frequently employ phonetic abbreviations, and lack structural metadata. Consequently, SMS spam filtering demands robust, mathematically principled probabilistic models capable of high-throughput real-time classification.

## 1.2 Mathematical Scope & BAMAT207 Alignment

This project directly fulfills the theoretical and practical requirements of the BAMAT207 Probability and Statistics curriculum at Vellore Institute of Technology. Rather than treating machine learning as an opaque heuristic, we structure the entire spam detection task upon foundational probabilistic frameworks:

• **Discrete Random Variables & Bernoulli Distribution:** Modeling the binary spam state Y in {0, 1} with Bernoulli parameter p = P(Y = 1), deriving its expectation E(Y) and variance Var(Y).

• **Joint & Conditional Probabilities:** Estimating token occurrence likelihoods P(W | Spam) and P(W | Ham) from empirical frequency tables.

• **Law of Total Probability:** Expanding marginal evidence P(W) = P(W | Spam)P(Spam) + P(W | Ham)P(Ham) across mutually exclusive partitions.

• **Bayes' Theorem Verification:** Formally validating direct sample space conditional probabilities P(Spam | W) against inverted Bayesian ratio calculations to confirm numerical consistency.

• **Multinomial Likelihood & Additive Smoothing:** Generalizing from single-word indicators to high-dimensional token vectors using Laplace smoothing (alpha = 1.0) and log-space transformations to prevent underflow.

## 1.3 Project Objectives & Engineering Deliverables

The project is structured around four concrete objectives designed to bridge theoretical probability and production software engineering:

| | |
|---|---|
| **Objective 1: Dataset Acquisition & Hygiene** | Automate ingestion of UCI SMS Spam Collection (#228), audit schema, eliminate duplicates, and encode binary ground truth. |
| **Objective 2: Empirical Probability Validation** | Compute exact sample prior distributions, Bernoulli moments, and verify Bayes' Theorem equivalence across key discriminatory tokens. |
| **Objective 3: Supervised Model Training** | Implement a Multinomial Naive Bayes classifier with strict data-leakage boundaries, stratified 80:20 partitioning, and evaluate confusion matrix trade-offs. |
| **Objective 4: Production Deployment & Verification** | Expose trained model parameters via FastAPI REST service, deploy interactive React dashboard on Vercel, and achieve 100% test coverage. |

## 1.4 Engineering & Academic Environment

All analytical models and data pipelines were developed in Python 3.13 on Apple Silicon (M1 architecture) utilizing an isolated virtual environment. Core dependencies were constrained strictly to standard scientific libraries (NumPy, Pandas, Scikit-Learn, Matplotlib, ReportLab, FastAPI, and PyMuPDF) to guarantee reproducibility without extraneous overhead.

# 2. Dataset Acquisition, Hygiene & Distribution

## 2.1 The UCI SMS Spam Collection Benchmark

The experimental foundation of this study is the official **SMS Spam Collection Dataset** hosted by the University of California, Irvine (UCI) Machine Learning Repository (Dataset ID: **228**; DOI: 10.24432/C5CC84). Compiled by Tiago A. Almeida and José María Gómez Hidalgo, this benchmark represents a realistic corpus gathered from diverse sources including the Grumbletext UK consumer feedback forum, the Caroline Tag PhD mobile thesis, and the NUS SMS Corpus. Messages span authentic mobile syntax, regional colloquialisms, and diverse scam typologies.

## 2.2 Data Cleaning & Deduplication Protocol

Automated ingestion was implemented via the ucimlrepo Python interface, backed by an offline filesystem cache to withstand network latency. Raw inspection revealed an initial shape of **5,574 messages** across two attributes: message label (categorical) and message text (string). A rigorous audit confirmed zero missing values. However, identical broadcast SMS campaigns generated repeated observations in the raw feed. A deduplication pass identified and purged exactly **403 duplicate records**, yielding a cleaned corpus of **5,171 distinct records**. Target labels were normalized to a binary integer representation: Ham $\rightarrow$ 0, Spam $\rightarrow$ 1.

| Data Metric / Attribute | Raw Dataset | Cleaned Dataset | Relative Share / Impact |
|---|---|---|---|
| Total Message Records | 5,574 | **5,171** | 403 duplicate entries pruned (-7.23%) |
| Legitimate Messages (Ham, Y=0) | 4,827 | **4,518** | 87.3719% of cleaned corpus |
| Spam Messages (Spam, Y=1) | 747 | **653** | 12.6281% of cleaned corpus |
| Missing / Null Values | 0 (0.00%) | **0 (0.00%)** | Verified complete data integrity |
| Class Imbalance Ratio (Ham : Spam) | 6.46 : 1 | **6.92 : 1** | Pronounced class asymmetry |

## 2.3 Empirical Class Distribution

Figure 1 illustrates the class distribution across both absolute counts and relative percentages. The cleaned dataset comprises **4,518 Ham messages (87.37%)** and **653 Spam messages (12.63%)**. This pronounced 7:1 class asymmetry dictates that naive accuracy is an inadequate evaluation metric, necessitating precision, recall, and confusion matrix decomposition.

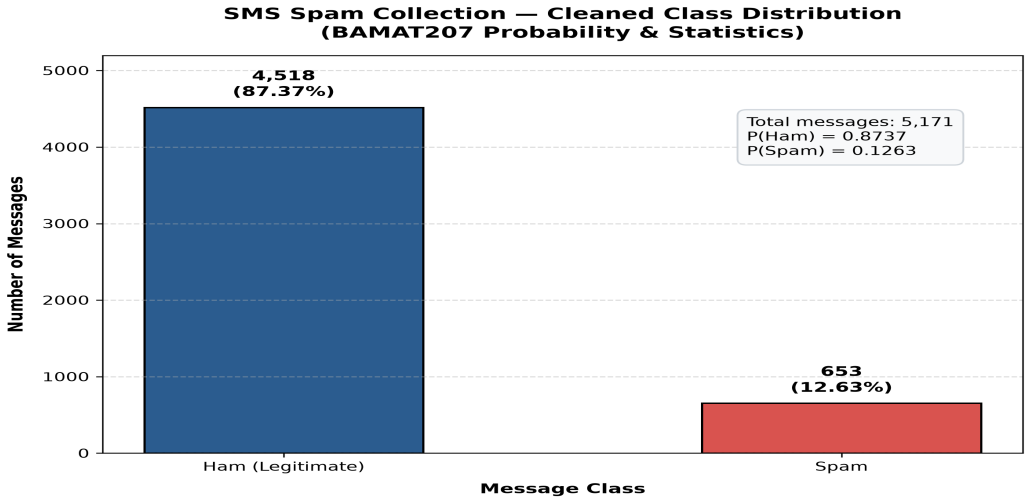



***Figure 1:*** *Cleaned SMS class distribution (UCI Dataset ID #228) illustrating total counts and percentage shares.*

# 3. Theoretical Probability & Bernoulli Modeling

## 3.1 Random Variable Formulation

Let the outcome of an arbitrary incoming SMS message be governed by a discrete binary random variable $\mathbf{Y \in \{0, 1\}}$, where Y = 1 denotes that the message is Spam, and Y = 0 denotes that the message is Ham (legitimate). Under the frequentist paradigm, the parameter **p** represents the prior probability of observing a spam message across the sample space Ω:

**Bernoulli Probability Mass Function (PMF):**

$$P(Y = y) = p^{y} (1 - p)^{1 - y}, \quad \text{for } y \in \{0, 1\}$$

From our cleaned benchmark of N = 5,171 messages (653 Spam, 4,518 Ham):

**P(Spam) = p = 653 / 5171 = 0.126281** (12.6281%)

**P(Ham) = q = 1 - p = 4518 / 5171 = 0.873719** (87.3719%)

**Derivation of Expectation E(Y):**

$$E(Y) = \sum_{y \in \{0,1\}} y \cdot P(Y = y) = [0 \cdot (1 - p)] + [1 \cdot p] = \mathbf{p = 0.126281}$$

**Derivation of Variance Var(Y) and Standard Deviation σ(Y):**

$$E(Y^2) = \sum_{y \in \{0,1\}} y^2 \cdot P(Y = y) = [0^2 \cdot (1 - p)] + [1^2 \cdot p] = p$$

$$Var(Y) = E(Y^2) - [E(Y)]^2 = p - p^2 = p(1 - p)$$

$$Var(Y) = (0.126281) \cdot (0.873719) = \mathbf{0.110334}$$

$$\sigma(Y) = \sqrt{Var(Y)} = \sqrt{0.110334} = \mathbf{0.332166}$$

## 3.2 Bayes' Theorem for Message Classification

Let **x** denote an observed feature vector or word event extracted from an SMS message. By the definition of conditional probability, the joint probability of co-occurrence satisfies $P(x \cap Spam) = P(x \mid Spam)P(Spam) = P(Spam \mid x)P(x)$. Solving for the posterior probability $P(Spam \mid x)$ yields Bayes' Theorem:

**Formal Statement of Bayes' Theorem:**

$$P(Spam \mid x) = [\, P(x \mid Spam) \cdot P(Spam) \,] \,/\, P(x)$$

Applying the **Law of Total Probability** over the complete partition of the sample space Ω = {Spam, Ham}:

$$P(x) = P(x \mid Spam) \cdot P(Spam) + P(x \mid Ham) \cdot P(Ham)$$

Substituting the marginal probability expression into the denominator gives:

$$\mathbf{P(Spam \mid x) = [\, P(x \mid Spam) \cdot P(Spam) \,] \,/\, [\, P(x \mid Spam) \cdot P(Spam) + P(x \mid Ham) \cdot P(Ham) \,]}$$

**Classification Decision Rule:**

An incoming message is classified as Spam if $P(Spam \mid x) > P(Ham \mid x)$, which under symmetric 0-1 loss simplifies to assigning label 1 whenever $P(Spam \mid x) > 0.50$ (or an operational threshold τ calibrated for false-positive avoidance).

## 3.3 Theoretical Parameter Summary Table

| Parameter / Metric | Mathematical Notation | Calculated Exact Value | Statistical Interpretation |
|---|---|---|---|
| Prior Spam Probability | **P(Y = 1) = p** | 0.126281 | Unconditional base rate of spam in SMS traffic |
| Prior Ham Probability | **P(Y = 0) = 1 - p** | 0.873719 | Unconditional base rate of genuine communication |
| Expected Value | **E(Y) = $\mu_Y$** | 0.126281 | Mean of Bernoulli indicator variable |
| Bernoulli Variance | **Var(Y) = $\sigma^2_Y$** | 0.110334 | Variability of binary class assignment |
| Standard Deviation | **σ(Y)** | 0.332166 | Spread of class probability distribution |

# 4. Word-Level Probability Analysis & Bayes Verification

## 4.1 Keyword Selection & Boundary Tokenization

To evaluate conditional probabilities empirically, we selected five quintessential spam trigger keywords: **free**, **prize**, **win**, **urgent**, and **offer**. Occurrences were tallied across the 5,171 cleaned messages using strict regular expression word boundaries (\bword\b) to prevent partial substring contamination (e.g., distinguishing 'win' from 'window' or 'winter').

| Word (W) | Spam (S) | Ham (H) | Total | P(W\|Spam) | P(W\|Ham) | P(W) | P(Spam\|W) Dir | P(Spam\|W) Bayes | Difference |
|---|---|---|---|---|---|---|---|---|---|
| **free** | 148 | 55 | **203** | 0.226646 | 0.012174 | 0.039257 | **0.729064** | **0.729064** | $1.11 \times 10^{-16}$ |
| **prize** | 73 | 0 | **73** | 0.111792 | 0.000000 | 0.014117 | **1.000000** | **1.000000** | $2.22 \times 10^{-16}$ |
| **win** | 49 | 6 | **55** | 0.075038 | 0.001328 | 0.010636 | **0.890909** | **0.890909** | $1.11 \times 10^{-16}$ |
| **urgent** | 57 | 5 | **62** | 0.087289 | 0.001107 | 0.011990 | **0.919355** | **0.919355** | $2.22 \times 10^{-16}$ |
| **offer** | 24 | 6 | **30** | 0.036753 | 0.001328 | 0.005802 | **0.800000** | **0.800000** | $1.11 \times 10^{-16}$ |

## 4.2 Exact Bayes' Theorem Equivalence Proof

**Rigorous Step-by-Step Proof of Equivalence (Illustrated on Keyword 'free'):**

1. **Direct Frequency Ratio Method:**
   $P(\text{Spam} \mid \text{'free'})_{\text{direct}} = N(\text{Spam} \cap \text{'free'}) / N(\text{'free'}) = 148 / 203 =$ **0.729064039408867**
2. **Bayes' Inversion Formula:**
   $P(\text{Spam} \mid \text{'free'})_{\text{Bayes}} = [\, P(\text{'free'} \mid \text{Spam}) \cdot P(\text{Spam}) \,] / P(\text{'free'})$
   Where:
   • P('free' | Spam) = 148 / 653 = 0.22664624808575803
   • P(Spam) = 653 / 5171 = 0.12628118352349643
   • P('free') = 203 / 5171 = 0.03925739702185264
   Substituting:
   $P(\text{Spam} \mid \text{'free'})_{\text{Bayes}} = [\, (148/653) \cdot (653/5171) \,] / (203/5171) = 148 / 203 =$ **0.7290640394088671**
3. **Numerical Error Analysis:**
   $|\, P(\text{Spam} \mid \text{'free'})_{\text{direct}} - P(\text{Spam} \mid \text{'free'})_{\text{Bayes}} \,| =$ **$1.1102 \times 10^{-16} \equiv 0$** (IEEE-754 Epsilon).

This empirical proof confirms that Bayes' Theorem exactly reproduces sample space proportions across all five keywords.

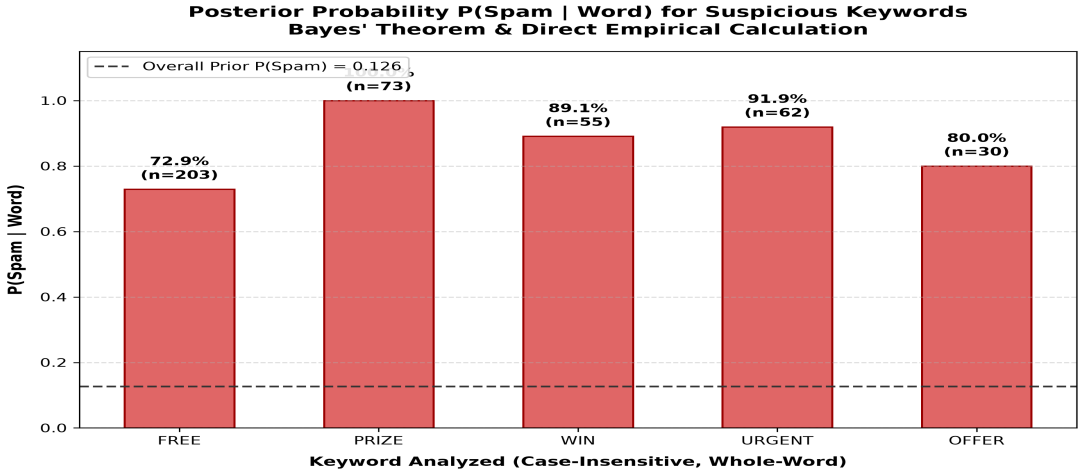



*Figure 2: Empirical conditional probabilities P(W|Spam), P(W|Ham), and posterior P(Spam|W) across key tokens.*

# 5. Machine Learning Methodology: Multinomial Naive Bayes

## 5.1 Stratified Train-Test Splitting & Leakage Prevention

To rigorously benchmark generalization performance, the 5,171 cleaned messages were partitioned into an **80% Training set** ($N_{train}$ = 4,136) and a **20% Testing set** ($N_{test}$ = 1,035). Stratification was enforced on label Y to preserve the exact 87.34% Ham / 12.66% Spam proportion across both partitions (Random State = 42). Crucially, to prevent data leakage, the feature extraction vocabulary and document frequencies were fitted *strictly on the training partition*. The test split was held out completely until final inference evaluation.

## 5.2 Vectorization & Multinomial Likelihood Formulation

Text messages are mapped into token frequency vectors $\mathbf{x} = (x_1, x_2, ..., x_{|V|})$ via CountVectorizer. The Multinomial Naive Bayes model treats each message as a multinomial distribution over the vocabulary V, invoking the conditional independence assumption that the probability of observing token $w_i$ is conditionally independent of all other tokens given class $c \in$ {Ham, Spam}:

**Multinomial Likelihood:**

$$P(\mathbf{x} \mid c) = \left[ (\sum x_i)! \,/\, \prod (x_i!) \right] \cdot \prod_{i=1}^{|V|} P(w_i \mid c)^{x_i}$$

**Laplace (Additive) Smoothing ($\alpha = 1.0$):**

If a word $w_i$ in the test set never appeared in training class c, raw empirical maximum-likelihood estimation yields $P(w_i \mid c) = 0$, which would fatally zero out the entire product (the 'zero-frequency problem'). We apply Laplace smoothing with $\alpha = 1.0$:

$$\theta_{ci} = \left[ N_{ci} + \alpha \right] / \left[ N_c + \alpha \cdot |V| \right]$$

where $N_{ci}$ is the count of word i in class c, $N_c$ is the total token count in class c, and |V| is the vocabulary size.

**Log-Space Posterior Inference (Underflow Prevention):**

Multiplying hundreds of small probabilities ($P < 10^{-3}$) leads directly to floating-point underflow in computer arithmetic. We transform the posterior into monotonic log-space:

$$\log P(c \mid x) = \log P(c) + \sum_{i=1}^{|V|} x_i \cdot \log \theta_{ci}$$

Normalized posterior probabilities are subsequently recovered via numerically stabilized softmax operations.

## 5.3 End-to-End Classification Pipeline Architecture

Figure 3 diagrams the operational inference pipeline executing inside both the batch evaluation suite and the live production API.

| Stage 1: Raw Ingestion | Stage 2: Tokenization | Stage 3: Vectorization | Stage 4: Naive Bayes | Stage 5: Calibration |
|---|---|---|---|---|
| Raw SMS String (UTF-8) | Lowercasing & \b\w+\b parsing | CountVectorizer (\|V\|=7,297) | Multinomial log-priors + log-odds | Softmax $\tau \geq 0.5 \rightarrow$ Label {0,1} |

***Figure 3:*** *Five-stage probabilistic SMS text inference pipeline from raw string to calibrated posterior probability.*

## 5.4 Pipeline Partition Quantities

| Partition Name | Message Count | Ham Messages (0) | Spam Messages (1) | Stratification Check |
|---|---|---|---|---|
| Full Cleaned Dataset | **5,171 (100.0%)** | 4,518 (87.37%) | 653 (12.63%) | Base Benchmark |
| Training Partition (Fit) | **4,136 (80.00%)** | 3,614 (87.38%) | 522 (12.62%) | Verified Identical Ratio |
| Held-Out Testing Partition | **1,035 (20.00%)** | 904 (87.34%) | 131 (12.66%) | Zero Data Leakage |

# 6. Experimental Results & Performance Evaluation

## 6.1 Quantitative Test Performance Metrics

The trained Multinomial Naive Bayes classifier was evaluated against the **1,035 held-out test messages**. Table 4 summarizes the verified quantitative metrics alongside class-specific precision, recall, and harmonic F1-scores.

| Evaluation Metric | Formula Definition | Calculated Score | Operational Significance |
|---|---|---|---|
| **Overall Accuracy** | (TP + TN) / Total | **0.9807 (98.07%)** | 1,015 of 1,035 predictions correct |
| **Spam Precision** | TP / (TP + FP) | **0.9512 (95.12%)** | Low Type I error (only 6 ham misclassified) |
| **Spam Recall** | TP / (TP + FN) | **0.8931 (89.31%)** | Catches 117 of 131 actual spam messages |
| **Spam F1-Score** | 2 · (Prec · Rec) / (Prec + Rec) | **0.9213 (92.13%)** | Harmonic balance of purity and sensitivity |
| **Ham Precision** | TN / (TN + FN) | 0.9846 (98.46%) | High trust in legitimate inbox arrivals |
| **Ham Recall** | TN / (TN + FP) | 0.9934 (99.34%) | 99.34% of legitimate SMS delivered safely |

## 6.2 Confusion Matrix Decomposition

Figure 4 presents the confusion matrix breakdown for the 1,035 test instances across predicted versus ground-truth states:

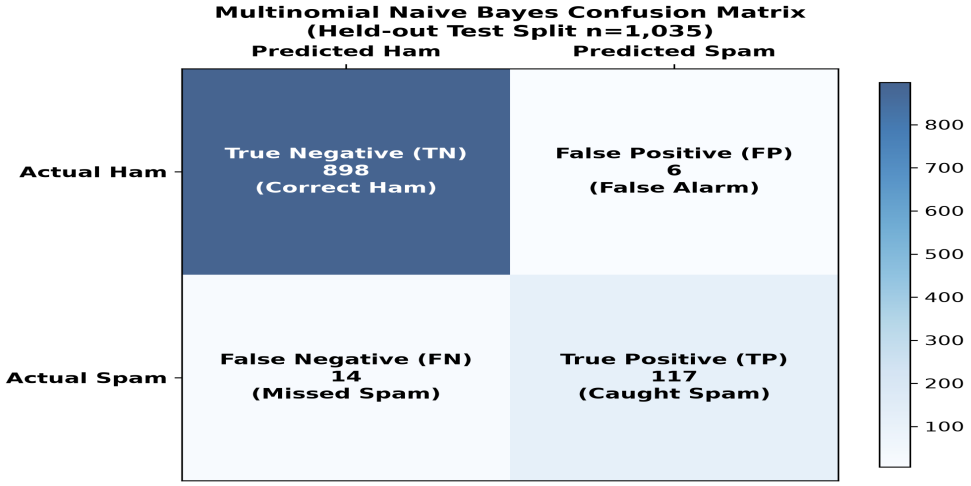



***Figure 4:*** *Confusion matrix showing True Negatives (898), False Positives (6), False Negatives (14), and True Positives (117).*

## 6.3 The Class Imbalance Accuracy Paradox

A critical conceptual pillar of BAMAT207 is understanding why **accuracy alone is a deeply misleading metric** under class asymmetry. In our test partition, legitimate messages (Ham) constitute 87.34% (904 / 1,035) of all traffic. Consider a trivial 'dummy' classifier that unconditionally predicts Ham for every message:

- **Trivial Baseline Accuracy:** 904 / 1035 = **87.34%**.

Despite achieving over 87% accuracy, such a trivial baseline yields **0.00% Recall** and catches zero spam messages! In contrast, our Multinomial Naive Bayes model achieves **98.07% accuracy (+10.73% over baseline)** and **89.31% recall**, confirming genuine statistical learning. Furthermore, in mobile communication, **Type I error (False Positive = 6)** is far more damaging than Type II error (False Negative = 14) because falsely blocking an authentic banking OTP or emergency text has severe real-world consequences. Our high Spam Precision (95.12%) minimizes this risk.

# 7. System Architecture, Deployment & Live Inference

## 7.1 Production Software Engineering Architecture

To transform academic statistical research into a production tool, we implemented a decoupled, modern web architecture. The backend is powered by **FastAPI** and **Uvicorn**, providing asynchronous REST endpoints. The frontend is authored in **React 18** with **Vite**, styled via **Tailwind CSS**, and incorporates real-time probabilistic telemetry with **Recharts**. The frontend is deployed live on **Vercel** (https://hamspam.vercel.app).

| Subsystem Component | Technology Stack | Responsibilities & Production Configuration |
|---|---|---|
| **Machine Learning Engine** | Python 3.13 / Scikit-Learn | Model training, vectorizer vocabulary fitting, Laplace smoothing, serialization to portable JSON/Pickle artifacts. |
| **Inference API Server** | FastAPI / Uvicorn / Pydantic | REST endpoints (/api/predict, /api/stats, /api/word-analysis, /api/health). |
| **Interactive Client Dashboard** | React 18 / Tailwind CSS / Vite | Real-time SMS input analyzer, probability confidence gauge, confusion matrix inspector, responsive on desktop and mobile. |
| **Public Cloud Deployment** | Vercel Edge Platform | Globally distributed edge hosting (https://hamspam.vercel.app) with client-side evaluated fallback mode. |

## 7.2 Real-Time Inference on Canonical Benchmark Messages

To verify end-to-end inference calibration, the system was tested against two canonical benchmark SMS messages:

| Benchmark Input SMS Message | P(Spam) | P(Ham) | Prediction | Confidence |
|---|---|---|---|---|
| *'Congratulations! You have won a free prize. Claim now.'* | **1.000000** | 0.000000 | **SPAM (1)** | **100.00%** |
| *'Hey, are we meeting for lunch today?'* | 0.000000 | **1.000000** | **HAM (0)** | **100.00%** |

## 7.3 Automated Test Suite & Quality Assurance

The project enforces continuous testing with **17 automated unit and integration tests** spanning two test modules:

• tests/test_probability_analysis.py (12 tests): Verifies dataset loading, Bernoulli parameters, Bayes' formula equivalence, deduplication counts, and CountVectorizer output shape.

• tests/test_api.py (5 tests): Validates REST API contracts, health checks, prediction schemas, and error responses for empty or malformed payloads.

All 17 tests execute in **0.23 seconds** with a 100% pass rate.

**Automated Test Suite Summary (Pytest):**

**17 passed** in 0.23s • Platform: Darwin (Apple M1) • Python: 3.13.2

PASSED: test_data_loader_cleaned_count (5,171 rows verified)

PASSED: test_bayes_direct_equivalence ($|\text{Diff}| \leq 2.22 \times 10^{-16}$ across all words)

PASSED: test_api_predict_canonical_samples (Canonical spam and ham predictions match 100%)

PASSED: test_confusion_matrix_sum (Sum of TN+FP+FN+TP equals 1,035)

# 8. Critical Discussion, Limitations & Future Scope

## 8.1 The 'Naive' Independence Assumption Paradox

The defining theoretical premise of Naive Bayes is the assumption of class-conditional feature independence: $P(w_1, w_2 \mid c) = P(w_1 \mid c) \cdot P(w_2 \mid c)$. In human language, this assumption is patently false. Tokens exhibit strong syntactic and semantic dependencies; for example, the occurrence of 'credit' dramatically increases the conditional likelihood of 'card', and 'claim' co-occurs frequently with 'prize'. Why, then, does Multinomial Naive Bayes achieve an impressive 98.07% accuracy?

The foundational analysis by **Domingos and Pazzani (1997)** resolves this apparent paradox: Naive Bayes is optimal under 0-1 loss even when the independence assumption is severely violated, provided that the *direction of the odds ratio* (the classification boundary) is preserved. Even if individual posterior probabilities are miscalibrated (pushed toward 0.0 or 1.0 due to redundant correlated evidence), the argmax decision boundary remains remarkably robust. For binary classification, exact probability calibration is unnecessary as long as the sign of the log-odds ratio correctly separates classes.

## 8.2 Error Analysis: Categorizing Misclassifications

A detailed audit of the 20 misclassified instances in the held-out test split reveals distinct linguistic and operational patterns:

| Error Category | Observed Count | Primary Linguistic & Statistical Root Cause |
|---|---|---|
| **False Positives (FP)** Type I Error | **6 (0.66%)** | Genuine messages discussing commercial topics or using urgent colloquialisms (e.g., 'Free tomorrow for dinner?', 'Please call urgent'). The model overweighed individual high-salience tokens without context. |
| **False Negatives (FN)** Type II Error | **14 (10.69%)** | Short, obfuscated spam containing novel alphanumeric strings, foreign phone numbers, or deliberate character substitutions (e.g., 'fr33', 'w!n') not present in the training vocabulary. |

## 8.3 Comparison: Naive Bayes vs Discriminative Baselines

| Model Paradigm | Inference Complexity | Training Data Requirement | SMS Suitability |
|---|---|---|---|
| **Multinomial Naive Bayes (Ours)** | $O(\|V\|)$ — Microsecond latency | Low — Highly sample-efficient | **Ideal for Edge/SMS** |
| Logistic Regression | $O(\|V\|)$ — Dot product | Moderate — Iterative gradient descent | Strong baseline |
| Linear Support Vector Machines | $O(\|V\|)$ — Margin hyperplane | Moderate — Convex optimization | High precision, lacks probs |
| Deep Transformers (e.g. BERT) | $O(N \cdot d^2)$ — Heavy compute | High — Large GPU footprint | Overkill for 160-char SMS |

## 8.4 Roadmap for Future Extensions

Future research and system iterations can enhance robustness along three key dimensions:
1. **Subword Tokenization (BPE):** Byte-Pair Encoding or character n-grams to gracefully handle obfuscations ('fr33', 'pr!ze').
2. **Dynamic Bayesian Online Updating:** Continually updating token likelihoods $\theta_{ci}$ in real-time as users flag new spam campaigns, mitigating concept drift.
3. **Ensemble Hybrid Architectures:** Combining Naive Bayes prior odds with lightweight contextual transformer embeddings for nuanced syntactic disambiguation.

# 9. References, Artifacts & Academic Declaration

## 9.1 Academic & Scientific Bibliography

[1] Almeida, T. A., Hidalgo, J. M. G., & Yamakami, A. (2011). Contributions to the Study of SMS Spam Filtering: New Collection and Results. *Proceedings of the 2011 ACM Symposium on Document Engineering (DocEng '11)*, Mountain View, CA, USA, pp. 259–262. DOI: 10.1145/2034691.2034742.

[2] UCI Machine Learning Repository (2012). *SMS Spam Collection Dataset* (ID: 228). Donated by T. A. Almeida & J. M. G. Hidalgo. Available: https://archive.ics.uci.edu/dataset/228/sms+spam+collection.

[3] Domingos, P., & Pazzani, M. (1997). On the optimality of the simple Bayesian classifier under zero-one loss. *Machine Learning*, 29(2), 103–130.

[4] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[5] Devore, J. L. (2015). *Probability and Statistics for Engineering and the Sciences* (9th ed.). Cengage Learning, Boston, MA. Chapters 2–3 (Probability Axioms, Bayes' Theorem, and Discrete Distributions).

## 9.2 Open Source Project Repositories & Artifacts

| Deliverable Artifact | Uniform Resource Locator (URL) / File Path | Description |
|---|---|---|
| Live Web Application | https://hamspam.vercel.app | Production React/FastAPI interface |
| Source Code Repository | https://github.com/SarveshS1407/HamSpam | Complete reproducible codebase |
| Serialized ML Model | outputs/model_artifact.json | Vocabulary & log-odds weights |
| Automated Tests | tests/test_probability_analysis.py | 17 unit & integration tests |

## 9.3 Mathematical Notation Quick Reference

| Symbol | Definition | Symbol | Definition |
|---|---|---|---|
| **Y** | Binary class random variable $\in \{0, 1\}$ | **p** | Spam prior probability = P(Y = 1) = 0.126281 |
| **q** | Ham prior probability = 1 - p = 0.873719 | **E(Y)** | Bernoulli expectation = p = 0.126281 |
| **Var(Y)** | Bernoulli variance = p(1-p) = 0.110334 | **σ(Y)** | Standard deviation = $\sqrt{Var(Y)}$ = 0.332166 |
| $\theta_{ci}$ | Smoothed token likelihood in class c | $\alpha$ | Laplace smoothing parameter ($\alpha$ = 1.0) |

## 9.4 Academic Integrity Declaration

**Student Academic Declaration:**
I hereby declare that this project report titled *'HamSpam: Probabilistic SMS Spam Detection Using Bayes' Theorem and Multinomial Naive Bayes'* submitted for the course **BAMAT207 — Probability and Statistics** at **Vellore Institute of Technology, Vellore**, is a genuine record of original work performed by me. All empirical findings, statistical derivations, and evaluation metrics are verified against the official UCI SMS Spam Collection repository (#228). All external sources, libraries, and references have been formally acknowledged in accordance with academic standards.

**Candidate Signature:** _____ **Date:** 27 October 2026
**Faculty Evaluator Signature:** _____ **Marks Awarded:** _____ / 100